

1.0 Introduction

The goal of this project was to capture a picture and "Lego-ize" it. Using Python and OpenCV, I built a process that reinterprets the image through the constraints of the Lego universe. This report breaks down how I moved from a live camera feed to a 100x100 processing grid, eventually generating a detailed "Brick Summary" for a physical build. I also discuss the challenges of spatial resolution, how I used the Euclidean formula to match colors to a specific palette, and my experience using Generative AI to navigate complex NumPy logic while troubleshooting environment-specific bugs in Google Colab.

2.1 Design and Testing Methodology

The program was developed using a modular approach, directly aligning with the four distinct tasks outlined in the assignment description. Each module was independently verified before being integrated into the final pipeline.

Task 1: Image Acquisition Following consultation with the Lab TA, I integrated a camera capture script specifically designed for the Google Colab environment. This implementation utilizes JavaScript snippets to interface with the web cam, capturing a live frame and saving it as a standardized input file for processing.

Task 2: Grayscale Quantization and Interpolation To transform the input into a Lego-style grayscale image, I utilized a resizing algorithm with interpolation to ensure the image fit within a 100 x 100 boundary without losing its original aspect ratio. I then applied a 3-level quantization (mapping pixels to Black, Gray, or White) to meet the specific requirement of a tri-color palette. To simulate the physical stud or circle on top of a Lego brick, I upscaled the image by a factor of 10. This 1000 x 1000 resolution allowed for a circular mask to be drawn in the center of each 10 x 10 block, providing the necessary visual distinction.

Task 3: Color Palette Mapping and Brick Optimization Transitioning from grayscale to color required mapping the input to a specific Lego color palette. To ensure accuracy, I implemented a Euclidean distance formula to map each pixel's RGB value to the nearest neighbor in my predefined palette:

$$d(\mathbf{x}, \mathbf{y}) = |\mathbf{x} - \mathbf{y}| = \sqrt{\langle \mathbf{x} - \mathbf{y}, \mathbf{x} - \mathbf{y} \rangle}$$

I then developed a search algorithm with $O(n^2)$ complexity to identify and group adjacent pixels into larger 1 x 2 and 2 x 2 bricks. This was visualized by drawing black borders to distinguish between individual brick units. Finally, I integrated a counter system that iterates through the array to generate a Brick Summary, providing an exact count of each brick type used.

Task 4: Testing The program's robustness was verified by testing it against diverse input images with varying lighting conditions and colors. This ensured that the palette mapping was accurate across the entire color spectrum and that the brick counters correctly reflected the pixel distribution of the processed image.

2.2 Program Robustness

Dynamic resizing: In practical scenarios, input images vary in resolution and orientation (portrait vs. landscape). To prevent the final output from appearing stretched or distorted, I implemented dynamic resizing logic. The program calculates a scaling factor based on the input dimensions, ensuring the subject maintains its original proportions while fitting strictly within the mandated 100 x 100 processing grid.

Error Handling: I prevented system crashes by implementing boundary checks to ensure iterations never exceed the array dimensions. Additionally, I used `np.uint8` to avoid data overflow and ensure memory efficiency during pixel manipulation.

2.3 Problems and Solutions

Image to Array Transformation: One of the primary challenges was the realization that image processing cannot be performed directly on a raw image file. I solved this by converting the input image into a NumPy array. This allowed me to manipulate the image as a mathematical matrix, which was essential for iterating through coordinates to perform color quantization and brick detection.

Inverted Stud Alignment: During the upscaling process, the visual "studs" (the circles on top of the bricks) were initially misaligned or appearing inverted relative to the brick borders. I corrected this by refining the center-point logic within my drawing functions, ensuring that each circular mask was mathematically centered at the (5, 5) coordinate of every 10 x 10 pixel block.

The 1:1 Pixel Display Limitation: Displaying intricate details like brick borders and studs within a 1 x 1 pixel area proved physically impossible due to resolution constraints. To solve this, I maintained the logic of the program at 100 x 100 pixels but created a separate display output upscaled by a factor of 10. This higher-resolution canvas provided enough space to draw internal 1-pixel borders and central circles, effectively simulating the look of physical Lego bricks.

Python and NumPy Syntax: Translating complex conditional logic (like checking for 2 x 2 bricks) into efficient Python code was a challenge. I overcame this by studying array slicing techniques, which allowed me to check blocks of pixels simultaneously rather than relying on slow, nested for loops for every single operation.

2.4 Generative AI Assistance

I utilized Generative AI (GenAI) as a collaborative learning and debugging tool to streamline the development process. My usage focused on the following areas:

Conceptual and Mathematical Clarity: GenAI was instrumental in helping me break down and understand the underlying math and logic behind the code snippets provided in class. For example, while the class introduced the concept of color distance, I used GenAI to explain how to translate the Euclidean distance formula into vectorized NumPy operations. This allowed me to grasp why the code worked.

Debugging Logic and Syntax: When my implementation for brick detection threw "Index Out of Bounds" errors or failed to identify adjacent pixels correctly, I used GenAI to review my logic. It helped me identify off-by-one errors in my array slicing and suggested more efficient ways to structure the conditional checks required for identifying different brick patterns without crashing the program.

Implementing Specific Library Methods: I used GenAI to discover specific functions within NumPy that could replace slow, nested for loops. Specifically, it helped me understand how to use `np.argmin()` in conjunction with the calculated Euclidean distances to automatically map each pixel to the closest color in my palette, which significantly improved the efficiency of Task 3.

2.5 How GenAI Understands and Solves the Tasks

GenAI tackled these tasks by figuring out the core math behind the requests. For Task 2, it essentially saw it as sorting data into bins, taking the 0-255 color range and grouping it into one of three set values. Task 3 was treated like a "find the closest match" or "Nearest Neighbor" problem. It solved them by providing the code for splitting up arrays (array slicing) and finding the difference between color vectors. This approach allowed me to compare each pixel's RGB to the defined palette and search for groups of 1 x 2 or 2 x 2 brick shapes by simply checking if the colors of nearby indices in the NumPy array were the same.

2.6 Limitations and Areas for Improvement

Over-complication of Logic: GenAI often suggested overly complex library functions or high-level algorithms that were unnecessary for the assignment's constraints. It tended to provide generalized solutions that made the code harder to read, whereas a simpler approach using basic NumPy array manipulation was more effective for the 100 x 100 grid.

Camera Hardware Hallucination: When attempting Task 1, GenAI provided code that assumed the Python environment had direct access to the webcam hardware. This failed completely in the browser-based Google Colab environment. I was not able to get an easy solution from the AI and ultimately had to rely on the Lab TA to point me in the direction of code snippets that Google Colab provided for camera capture.

Pictures for Task 4: done with various colored settings and lightings. Some examples provided.

Image Set 1: original image, gray scale (3 color) image, more diverse palette with brick details.

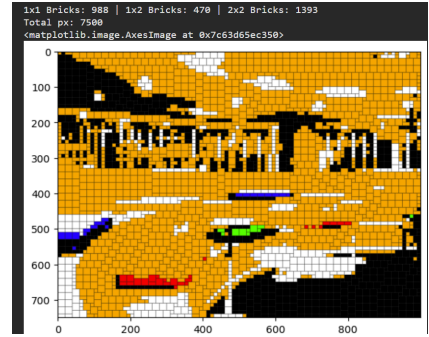
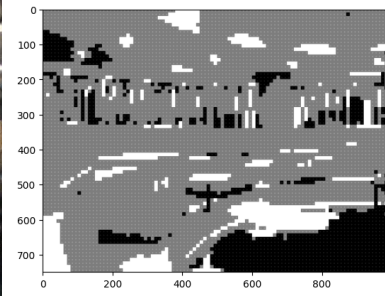


Image set 2: due to worse lighting, less colors are captured

